

FIȘĂ DE LUCRU — TEME

Python · List vs Tuple vs Set · Clasa a X-a

Nume: _____ Clasa: _____ Data: _____

Exercițiul 1 — Construirea celor trei tipuri de colecții

Folosind literele ['a', 'b', 'c', 'a'], completează codul de mai jos:

- Creează o **listă** numită `letters_list` care să păstreze toate elementele în ordine.
- Creează un **tuple** numit `letters_tuple` cu aceleași elemente, în ordine.
- Creează un **set** numit `letters_set` care să conțină fiecare literă o singură dată.

```
source_letters = ['a', 'b', 'c', 'a']

letters_list = ...
letters_tuple = ...
letters_set = ...
```

Atenție: seturile elimină automat duplicatele și nu păstrează ordinea elementelor.

Exercițiul 2 — Obținerea valorilor unice folosind un set

Scrie o funcție numită `unique_values(nums)` care primește o listă de numere și returnează un set cu valorile unice.

Sugestie: Convertirea unei liste într-un set elimină automat duplicatele. Folosește `set()`.

```
def unique_values(nums):
    # Codul tău aici
    ...
```

Exercițiul 3 — Indexare și ordine

Implementează cele două funcții de mai jos:

- `third_item(seq)` — returnează elementul de la indexul 2 dintr-o listă sau tuple.
- `try_index_set(s)` — încearcă să acceseze `s[0]`. Dacă apare `TypeError`, returnează șirul `'TypeError'`; altfel returnează valoarea.

```
def third_item(seq):
    # returnează elementul de la index 2
    ...

def try_index_set(s):
    # încearcă s[0]; returnează 'TypeError' dacă e cazul
    ...
```

Scopul: Seturile nu permit accesarea elementelor prin index — spre deosebire de liste și tuple.

Exercițiul 4 — Mutabilitatea în acțiune

Implementează cele două funcții:

`mutate_list(letters)`

- Adaugă (append) litera 'd'
- Elimină (remove) litera 'b'
- Reatribuie elementul de la indexul 0 cu valoarea 'z'
- Returnează lista modificată

`attempt_tuple_change(t)`

- Încearcă să adauge '!' la tuplul `t`
- Dacă apare `TypeError`, returnează un tuplu de forma `(t, 'TypeError')`
- Altfel, returnează noul obiect

```
def mutate_list(letters):  
    # modifică lista conform descrierii și returnează lista  
    ...  
  
def attempt_tuple_change(t):  
    # încearcă să modifice un tuplu; în caz de TypeError returnează (t,  
    'TypeError')  
    ...
```

Tuplurile sunt imuabile — odată create, elementele lor nu pot fi modificate. Listele sunt mutabile.

Exercițiul 5 — Alegerea tipului corect de colecție

Implementează cele două funcții de mai jos:

- `birthday_record(name, y, m, d)` — returnează un tuplu de forma `(name, (y, m, d))` ce stochează ziua de naștere a unui prieten.
- `replace_in_set(s, old, new)` — înlocuiește un element dintr-un set: elimină `old` și adaugă `new`. Returnează setul modificat.

```
def birthday_record(name, y, m, d):  
    # codul tău  
    ...  
  
def replace_in_set(s, old, new):  
    # codul tău  
    ...
```

De ce tuplu pentru zile de naștere? Sunt imuabile — protejează datele de modificări accidentale.

★ Exerciții suplimentare — Metode ale colecțiilor ★

Exercițiul 6 — Metode ale listei — append, insert, pop, sort

Pornind de la lista `fructe = ['mar', 'banana', 'cireș']`, realizează operațiunile de mai jos în ordine și afișează lista după fiecare pas:

- Adaugă `'portocala'` la sfârșitul listei cu `append()`
- Inserează `'ananas'` pe poziția 1 cu `insert()`
- Elimină ultimul element cu `pop()` și salvează valoarea returnată într-o variabilă
- Sortează lista alfabetic cu `sort()` și afișează rezultatul

```
fructe = ['mar', 'banana', 'cireș']

# 1. append

# 2. insert

# 3. pop
eliminat = ...

# 4. sort si print
```

Exercițiul 7 — Metode ale setului — add, discard, union, intersection

Lucrează cu seturile de mai jos și completează codul:

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
```

- Adaugă elementul 7 în setul `a` cu metoda `add()`
- Elimină elementul 2 din setul `a` cu `discard()` (fără eroare dacă nu există)
- Calculează reuniunea dintre `a` și `b` cu `union()`
- Calculează intersecția dintre `a` și `b` cu `intersection()`

```
# 1. add

# 2. discard

reuniune = ...
intersecție = ...
print(reuniune)
print(intersecție)
```

Exercițiul 8 — Metode utile — count, index, len

Folosind lista și tuplul de mai jos, răspunde la întrebările cu ajutorul metodelor potrivite:

```
note = [7, 9, 8, 10, 9, 7, 9, 6]
culori = ('rosu', 'verde', 'albastru', 'verde', 'galben')
```

- De câte ori apare nota 9 în lista **note**? Folosește **count()**
- Pe ce index se află prima apariție a lui 'verde' în tuplu? Folosește **index()**
- Câte elemente are lista **note**? Folosește **len()**
- Care este nota maximă și cea minimă din lista **note**? Folosește **max()** și **min()**

```
aparitii_9      = ...
index_verde     = ...
numar_elemente = ...
nota_max       = ...
nota_min       = ...

print(aparitii_9, index_verde, numar_elemente, nota_max, nota_min)
```

Exercițiul 9 — Conversii între colecții

Completează funcțiile de mai jos care convertesc colecții dintr-un tip în altul:

- **list_to_set(lst)** — primește o listă și returnează un set (fără duplicate)
- **set_to_sorted_list(s)** — primește un set și returnează o listă sortată
- **list_to_tuple(lst)** — primește o listă și returnează un tuplu cu aceleași elemente

```
def list_to_set(lst):
    ...

def set_to_sorted_list(s):
    ...

def list_to_tuple(lst):
    ...

# Testează funcțiile:
print(list_to_set([1, 2, 2, 3, 3]))      # {1, 2, 3}
print(set_to_sorted_list({5, 1, 3}))    # [1, 3, 5]
print(list_to_tuple(['x', 'y', 'z']))    # ('x', 'y', 'z')
```

Funcțiile built-in **set()**, **list()**, **sorted()**, **tuple()** realizează conversiile între tipuri.

Exercițiul 10 — Problemă de sinteză — Analiza unei clase

Clasa a IX-a A are următoarele note la matematică (unii elevi au lipsit și au note înregistrate de mai multe ori):

```
note_clasa = [5, 8, 7, 10, 8, 9, 5, 6, 10, 7, 8, 9, 7, 6, 5]
```

- Găsește câte **note distincte** există (folosește un set)
- Determină nota **maximă** și **minimă** din lista de note
- De câte ori apare nota 8 în listă?

- Creează un tuplu cu notele sortate crescător (folosind `sorted()` și `tuple()`)

```
note_clasa = [5, 8, 7, 10, 8, 9, 5, 6, 10, 7, 8, 9, 7, 6, 5]

# 1. note distincte
distincte = ...
print('Note distincte:', len(distincte))

# 2. max si min
print('Max:', ..., ' Min:', ...)

# 3. aparitii nota 8
print('Aparitii 8:', ...)

# 4. tuplu sortat
note_sorted = ...
print('Tuplu sortat:', note_sorted)
```